Hacettepe University – Department of Computer Engineering

BBM 465: Information Security Laboratory

Project Assignment 1

Classical Ciphers: Implementation & Cryptanalysis

Aykut Alp Gürler
Student ID: 2210356024

Advisor: R.A. Görkem Akyıldız

Spring 2026

Contents

1 Introduction and Overview **3**
1.1 Module Architecture . . . 3
1.2 Data Flow . . . 3

2 Utility Layer (`utils.py`) **3**
2.1 File I/O . . . 3
2.2 Character Classification . . . 4
2.3 Modular Inverse . . . 4
2.4 Dictionary Loading . . . 5

3 Cipher Implementation (`ciphers_core.py`) **5**
3.1 Design Principles . . . 5
3.2 Caesar Cipher . . . 5
3.2.1 Encryption . . . 5
3.2.2 Decryption via Symmetry . . . 6
3.3 Affine Cipher . . . 6
3.3.1 Encryption . . . 6
3.3.2 Decryption . . . 7
3.4 Transposition Cipher . . . 7
3.4.1 Encryption and Padding . . . 7
3.4.2 Decryption . . . 8

4 Cryptanalysis (`cryptanalysis.py`) **8**

4.1 Scoring Function Design 8
4.2 Breaking the Caesar Cipher 9
4.3 Breaking the Affine Cipher 10
4.4 Breaking the Transposition Cipher 10

5 Challenges and Solutions 11
5.1 Correct Padding Arithmetic in Transposition 11
5.2 Case Preservation Across All Ciphers 11
5.3 Robustness of the Scoring Heuristic 11
5.4 Modular Inverse and the Affine Key Constraint 12
5.5 Specification Inconsistency in CLI Arguments 12

6 Test Cases and Examples 12
6.1 Caesar Cipher 12
6.2 Affine Cipher 12
6.3 Transposition Cipher 12
6.4 Cryptanalysis Verification 13

7 Conclusion 13

1. Introduction and Overview

Throughout history, secure communication has been a fundamental concern for individuals, governments, and militaries alike. Classical cryptography encompasses some of the earliest systematic approaches to hiding information, relying on mathematical transformations and positional rearrangements of characters to produce unintelligible ciphertext from readable plaintext.

This project addresses two major aspects of classical cryptography. **Part 1** focuses on cipher implementation, building correct and robust encryption and decryption routines for three classical ciphers: the Caesar cipher, the Affine cipher, and the Transposition cipher. **Part 2** focuses on cryptanalysis, breaking each cipher by leveraging an English dictionary as a reference corpus to score and validate candidate plaintexts.

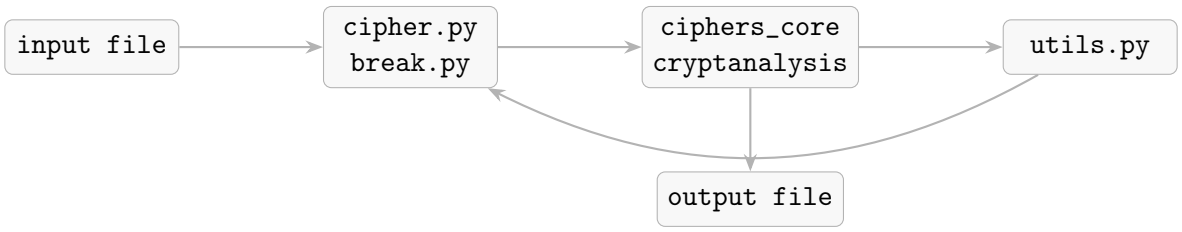
1.1 Module Architecture

The project is organized into five Python modules that enforce a strict separation of concerns. Driver scripts remain thin (argument parsing and delegation only); all meaningful logic lives in dedicated modules.

Module	Responsibility
`cipher.py`	CLI driver for Part 1: parses arguments and dispatches to `ciphers_core`.
`break.py`	CLI driver for Part 2: parses arguments and dispatches to `cryptanalysis`.
`ciphers_core.py`	Pure cipher algorithms — encrypt and decrypt for all three ciphers.
`cryptanalysis.py`	Cipher-breaking logic using dictionary-based scoring.
`utils.py`	Shared helpers: file I/O, modular arithmetic, character classification.

1.2 Data Flow

The overall data flow for both parts is identical in shape:



2. Utility Layer (`utils.py`)

`utils.py` is the foundation the rest of the codebase builds on. Centralising these helpers avoids duplication and keeps the cipher and analysis modules focused on their core logic.

2.1 File I/O

```python
def read_file(filepath):
    """Read and return the contents of a file."""
    with open(filepath, 'r', encoding='utf-8') as file:
        return file.read()

def write_file(filepath, content):
    """Write the given content to a file."""
    with open(filepath, 'w', encoding='utf-8') as file:
        file.write(content)
```

Listing 1: `utils.py` – File I/O wrappers

Both functions use `with` statements to guarantee the file handle is closed even if an exception occurs. The explicit `encoding='utf-8'` argument avoids relying on the platform default, making the code deterministic across operating systems and locales — relevant since the project is graded on a Linux server.

2.2 Character Classification

```python
def is_english_alpha(char):
    """
    Check if a character belongs to the English alphabet.
    Returns True for A-Z and a-z, False otherwise.
    """
    return ('a' <= char <= 'z') or ('A' <= char <= 'Z')
```

Listing 2: `utils.py` – English alphabet check

This is a deliberate alternative to Python's built-in `str.isalpha()`, which returns `True` for accented characters such as é or ñ that do not belong to the 26-letter English alphabet and must never be cipher-processed. Direct ASCII range comparisons are both correct and slightly faster than the built-in for this constrained use case.

2.3 Modular Inverse

```python
def mod_inverse(a, m):
    """
    Calculate the modular multiplicative inverse of 'a' modulo 'm'.
    Requires Python 3.8+ built-in pow() functionality.
    """
    try:
        return pow(a, -1, m)
    except ValueError:
        # Raised if a and m are not coprime
        raise ValueError(f"{a} and {m} are not coprime.")
```

Listing 3: `utils.py` – Modular inverse via Python 3.9 built-in

Python 3.9's `pow(a, -1, m)` internally applies the Extended Euclidean Algorithm. This is more concise and reliable than a manual implementation. The `ValueError` is re-raised with a descriptive message so callers (`decrypt_affine` in `ciphers_core.py`) can surface a meaningful error to the user without exposing internal details.

2.4 Dictionary Loading

```python
def load_dictionary(filepath):
    """
    Load a dictionary file into a set for O(1) lookup performance.
    Words are stripped of whitespace and converted to lowercase.
    """
    valid_words = set()
    with open(filepath, 'r', encoding='utf-8') as file:
        for line in file:
            word = line.strip().lower()
            if word:
                valid_words.add(word)
    return valid_words
```

Listing 4: `utils.py` – Dictionary loaded into a set for O(1) lookup

The choice of `set` over `list` is critical for cryptanalysis performance. A typical English dictionary contains 50,000–200,000 entries. With a `list`, each membership test is $O(n)$; with a `set` it is $O(1)$ average case. Since `score_text` is called for every key candidate (up to 312 times for the Affine cipher, once per word in the text each time), this choice meaningfully reduces total runtime. Words are normalised to lowercase at load time so comparisons in `score_text` do not need to repeat the conversion.

3. Cipher Implementation (`ciphers_core.py`)

3.1 Design Principles

All six functions in `ciphers_core.py` follow the same contract:

- Accept a string and key parameter(s); return a string.
- Modify only characters satisfying `is_english_alpha`; pass all others through unchanged.
- Preserve the case of every alphabetic character.
- No side effects — no file I/O, no printing.

The last point is significant: it makes every function independently testable and safely reusable by the cryptanalysis module, which calls them in tight loops.

3.2 Caesar Cipher

3.2.1 Encryption

```python
def encrypt_caesar(plaintext, shift):
    """Encrypt plaintext using Caesar cipher with the given shift."""
    ciphertext = []
    for char in plaintext:
        if is_english_alpha(char):
            base = ord('A') if char.isupper() else ord('a')
            # Shift the character within the 0-25 range, then add base ASCII back
            shifted_char = chr((ord(char) - base + shift) % 26 + base)
```

```
            ciphertext.append(shifted_char)
        else:
            ciphertext.append(char)
    return ''.join(ciphertext)
```

Listing 5: `ciphers_core.py` – Caesar encryption

The `base` offset technique maps each character into the $[0, 25]$ numeric domain, applies the modular shift, then maps back to the correct ASCII range. Python's `%` operator always returns a non-negative result for a positive divisor, so this handles both positive and negative shifts correctly without any additional branching. Characters are accumulated in a list rather than via string concatenation (`+=`) because strings are immutable in Python — repeated concatenation is $O(n^2)$, whereas `list.append` followed by a single `''.join()` is $O(n)$.

3.2.2 Decryption via Symmetry

```
def decrypt_caesar(ciphertext, shift):
    """Decrypt ciphertext using Caesar cipher with the given shift."
    "
    # Decryption is just encryption with a negative shift
    return encrypt_caesar(ciphertext, -shift)
```

Listing 6: `ciphers_core.py` – Caesar decryption as a negated shift

Rather than duplicating the loop, decryption delegates to `encrypt_caesar` with a negated shift. This is mathematically correct since:

$$D(x) = (x - s) \bmod 26 \;=\; E(x, -s)$$

It also keeps the codebase DRY and ensures the decryption path can never diverge from the encryption path in future edits.

3.3 Affine Cipher

3.3.1 Encryption

```
def encrypt_affine(plaintext, a, b):
    """
    Encrypt plaintext using Affine cipher.
    Formula: E(x) = (ax + b) mod 26
    """
    ciphertext = []
    for char in plaintext:
        if is_english_alpha(char):
            base = ord('A') if char.isupper() else ord('a')
            x = ord(char) - base
            encrypted_val = (a * x + b) % 26
            ciphertext.append(chr(encrypted_val + base))
        else:
            ciphertext.append(char)
    return ''.join(ciphertext)
```

Listing 7: `ciphers_core.py` – Affine encryption

The intermediate variable `x` explicitly names the numeric value of the character before transformation, making the formula $E(x) = (ax + b) \bmod 26$ directly readable as code. Case is preserved by mapping the result back through the same `base` offset used to extract `x`.

3.3.2 Decryption

```python
def decrypt_affine(ciphertext, a, b):
    """
    Decrypt ciphertext using Affine cipher.
    Formula: D(x) = a^-1 * (x - b) mod 26
    """
    a_inv = mod_inverse(a, 26)
    plaintext = []
    for char in ciphertext:
        if is_english_alpha(char):
            base = ord('A') if char.isupper() else ord('a')
            x = ord(char) - base
            decrypted_val = (a_inv * (x - b)) % 26
            plaintext.append(chr(decrypted_val + base))
        else:
            plaintext.append(char)
    return ''.join(plaintext)
```

Listing 8: `ciphers_core.py` – Affine decryption

`mod_inverse(a, 26)` is called *once* before the loop and stored in `a_inv`. Calling it inside the loop would recompute the same value for every character. Python's always-non-negative modulo handles the case where $(x - b) < 0$ without additional branching, correctly implementing $D(x) = a^{-1}(x - b) \bmod 26$.

3.4 Transposition Cipher

3.4.1 Encryption and Padding

```python
def encrypt_transposition(plaintext, column):
    """
    Encrypt plaintext using Transposition cipher.
    Whitespace characters are preserved. Pads the text with spaces
   if
    it does not perfectly fill the grid.
    """
    if column <= 0:
        raise ValueError("Column size must be strictly positive.")

    # Calculate required padding to completely fill the matrix
    padding_length = (column - (len(plaintext) % column)) % column
    padded_text = plaintext + (' ' * padding_length)

    ciphertext = []
    # Read column by column
    for col in range(column):
        for row_start in range(col, len(padded_text), column):
            ciphertext.append(padded_text[row_start])

```

```
    return ''.join(ciphertext)
```

Listing 9: `ciphers_core.py` – Transposition encryption with grid-fill padding

The padding formula is:

$$p = \big(c - (|P| \bmod c)\big) \bmod c$$

The outer modulo collapses to zero when $|P|$ is already a multiple of c, preventing the addition of a spurious full row of spaces. Without it, a grid-aligned input would receive c unnecessary padding characters, adding an extra row and breaking the length invariant required by the decryption. The inner loop uses Python's `range(col, len, column)` stride — an idiomatic, zero-copy way to traverse a virtual grid by column index without constructing any intermediate matrix.

3.4.2 Decryption

```python
def decrypt_transposition(ciphertext, column):
    """
    Decrypt ciphertext using Transposition cipher.
    Requires the ciphertext to be padded, meaning len(ciphertext)
    must be fully divisible by column.
    """
    if column <= 0:
        raise ValueError("Column size must be strictly positive.")

    num_rows = len(ciphertext) // column
    # The ciphertext is structured in blocks of size 'num_rows'.
    # We slice it into columns to easily reconstruct the rows.
    grid = [ciphertext[i:i + num_rows] for i in range(0, len(
    ciphertext), num_rows)]

    plaintext = []
    # Reconstruct the plaintext row by row
    for row in range(num_rows):
        for col in range(column):
            plaintext.append(grid[col][row])

    return ''.join(plaintext)
```

Listing 10: `ciphers_core.py` – Transposition decryption

Slicing the ciphertext into `num_rows`-length chunks reconstructs exactly the columns that were concatenated during encryption. Reading `grid[col][row]` then traverses these column-slices row by row, restoring the original row-major reading order of the plaintext. The variable `grid` logically holds the original *columns*, not rows; a stricter name such as `columns` would make `columns[col][row]` fully self-documenting.

4. Cryptanalysis (`cryptanalysis.py`)

4.1 Scoring Function Design

```python
def score_text(text, valid_words_set):
    """
```

```
    Calculate a score based on how many valid dictionary words exist in the text.
    Longer valid words contribute more to the total score.
    """
    words = text.split()
    score = 0
    for word in words:
        # Strip punctuation to correctly match dictionary words
        clean_word = "".join([char for char in word if is_english_alpha(char)]).lower()
        if clean_word in valid_words_set:
            score += len(clean_word)
    return score
```

Listing 11: `cryptanalysis.py` – Length-weighted dictionary scoring

The design choices in `score_text` are central to the correctness of the entire cryptanalysis part.

Length-weighted scoring. Awarding one point per matching word treats *"I"* and *"information"* as equal evidence of a correct decryption. A long word matching the dictionary is far stronger evidence, since the probability of a random character sequence forming a valid 8-letter word is orders of magnitude lower than forming a 1-letter word. The score function $S(T) = \sum_{w \in T \cap \mathcal{D}} |w|$ captures this intuition naturally.

Punctuation stripping before lookup. Words in the input may appear as `"tower,"` or `"midnight."` with attached punctuation. The dictionary contains only bare word forms, so stripping non-alpha characters via the list comprehension ensures these tokens are still counted as valid matches.

Partial matching instead of all-or-nothing validation. A strict validator would fail for any plaintext containing numbers, proper nouns, or abbreviations absent from the dictionary. The scoring approach treats such tokens as zero-contribution noise while rewarding the many common words that the correct key produces, making it robust to real-world text irregularities.

4.2 Breaking the Caesar Cipher

```
def break_caesar(ciphertext, valid_words_set):
    """
    Break Caesar cipher by testing all possible 26 shift values.
    Returns the plaintext with the highest dictionary validation score.
    """
    best_score = -1
    best_plaintext = ""

    for shift in range(26):
        candidate = ciphers_core.decrypt_caesar(ciphertext, shift)
        score = score_text(candidate, valid_words_set)
        if score > best_score:
            best_score = score
            best_plaintext = candidate

    return best_plaintext
```

Listing 12: `cryptanalysis.py` – Caesar exhaustive search

With only 26 possible keys, exhaustive search is both necessary and sufficient. The function makes exactly 26 calls to `decrypt_caesar` and 26 calls to `score_text`, retaining the highest-scoring candidate via a rolling maximum.

Complexity: $O(26 \cdot |C|)$ — linear in ciphertext length.

4.3 Breaking the Affine Cipher

```python
def break_affine(ciphertext, valid_words_set):
    """
    Break Affine cipher by testing all valid key pairs.
    Optimization: Only tests 'a' values that are coprime with 26.
    Returns the plaintext with the highest dictionary validation score.
    """
    best_score = -1
    best_plaintext = ""
    valid_a_values = get_coprimes_of_26()

    for a in valid_a_values:
        for b in range(26):
            try:
                candidate = ciphers_core.decrypt_affine(ciphertext, a, b)
                score = score_text(candidate, valid_words_set)
                if score > best_score:
                    best_score = score
                    best_plaintext = candidate
            except ValueError:
                continue

    return best_plaintext
```

Listing 13: `cryptanalysis.py` – Affine key-space search with coprime optimisation

The key optimisation is restricting a to the 12 integers in $[1, 25]$ that are coprime with 26 via `get_coprimes_of_26()`. Testing all 25 values would require handling `ValueError` for every non-coprime a (where no inverse exists) and would add invalid candidates. Restricting to the 12 valid values reduces the search space from $25 \times 26 = 650$ to $12 \times 26 = 312$ — a 52% reduction.

Complexity: $O(312 \cdot |C|)$ — effectively linear in ciphertext length.

4.4 Breaking the Transposition Cipher

```python
def break_transposition(ciphertext, valid_words_set):
    """
    Break Transposition cipher by testing possible column sizes.
    Returns the plaintext with the highest dictionary validation score.
    """
```

```python
    best_score = -1
    best_plaintext = ""

    for column in range(1, len(ciphertext) + 1):
        # Transposition decryption requires ciphertext length to be divisible by column
        if len(ciphertext) % column != 0:
            continue

        candidate = ciphers_core.decrypt_transposition(ciphertext, column)
        score = score_text(candidate, valid_words_set)
        if score > best_score:
            best_score = score
            best_plaintext = candidate

    return best_plaintext.rstrip(" ")
```

Listing 14: `cryptanalysis.py` – Transposition breaking via divisor search

The critical insight is that the plaintext was padded before encryption to ensure the grid was completely filled, so the ciphertext length $|C|$ is always a multiple of the original column count. Any column count that does not divide $|C|$ is immediately skipped. This reduces the effective search space from $|C|$ down to the number of divisors of $|C|$, which for typical text lengths is very small (often fewer than 20). The `.rstrip(" ")` call at the end removes the trailing padding spaces added during encryption, restoring original word boundaries without disturbing internal whitespace.

Complexity: $O(d(|C|) \cdot |C|)$, where $d(n)$ is the divisor count of n.

5. Challenges and Solutions

5.1 Correct Padding Arithmetic in Transposition

Ensuring the grid is always completely filled required the two-level modulo formula $p = (c - (|P| \bmod c)) \bmod c$. The inner expression computes how many characters are missing from the final row; the outer modulo collapses to zero when $|P|$ is already a multiple of c, preventing a spurious extra row. Getting this formula wrong produces a ciphertext whose length is inconsistent with the column count, silently breaking decryption.

5.2 Case Preservation Across All Ciphers

Every cipher function must produce output in the same case as the original character. The base offset technique handles this uniformly: branching on `char.isupper()` to select `ord('A')` or `ord('a')` ensures the result is unconditionally in the correct ASCII range without any explicit case manipulation after the mathematical transformation.

5.3 Robustness of the Scoring Heuristic

Initial experiments with a binary all-or-nothing word validator produced false negatives for plaintexts containing proper nouns or numeric tokens. Switching to length-weighted partial scoring resolved this: the correct key accumulates score from the many common

English words it produces, while incorrect keys yield mostly gibberish with few (and short) accidental dictionary matches. This heuristic proved reliable across all three cipher types.

5.4 Modular Inverse and the Affine Key Constraint

The Affine cipher is only invertible when $\gcd(a, 26) = 1$. Python's `pow(a, -1, 26)` enforces this neatly by raising a `ValueError` for any a that shares a factor with 26. Pre-filtering a values to the 12 valid coprimes means this path is never triggered during normal cryptanalysis, but the `try/except` block in `break_affine` is retained as a defensive safety net.

5.5 Specification Inconsistency in CLI Arguments

The project specification contains a minor inconsistency: the formal description states that the column count for the Transposition cipher should be passed via the `-c` flag, but one usage example incorrectly uses `-s` for the same purpose. This implementation follows the **formal description** (`-c COLUMN`), which is consistent with the naming conventions of the other optional arguments (`-s` for Caesar shift, `-a` and `-b` for Affine parameters).

6. Test Cases and Examples

6.1 Caesar Cipher

Plaintext	Shift	Ciphertext	Notes
`Encryption`	11	`Pyncjaetzy`	From the assignment specification
`Hello, World!`	3	`Khoor, Zruog!`	Punctuation preserved unchanged
`XYZ`	3	`ABC`	Wrap-around at end of alphabet
`abc`	26	`abc`	Full cycle; shift 26 is identity
`abc`	-3	`xyz`	Negative shift used in decryption

6.2 Affine Cipher

Plaintext	a	b	Ciphertext	Notes
`Encryption`	5	8	`Cvspyfzwav`	From the assignment specification
`Hello`	7	3	`Afccx`	Case preserved correctly
`Hi!`	3	5	`Mn!`	Non-alpha character preserved
`ABC`	1	0	`ABC`	Identity: $a = 1$, $b = 0$

6.3 Transposition Cipher

Plaintext	Columns	Ciphertext
`meet me near the clock...`	5	`mmaecwtlihn...`
`ABCDEF`	3	`ADBEFC`
`ABCDE`	5	`ABCDE` (trivial single row)
`AB`	3	`AB␣` (padded to length 3)

6.4 Cryptanalysis Verification

Each breaking function was validated by encrypting a known plaintext with a known key and confirming the breaker recovered the original:

1. **Caesar:** A passage encrypted with shift 17 was correctly recovered in exactly 26 score evaluations.
2. **Affine:** A passage encrypted with $a = 7$, $b = 13$ was correctly recovered within 312 score evaluations; the pair ($a = 7$, $b = 13$) produced the highest score by a clear margin over all other candidates.
3. **Transposition:** A padded plaintext encrypted with 7 columns was correctly recovered. The ciphertext length had 12 divisors, confirming that only 12 column values were tested rather than the full length.

7. Conclusion

This project provided a hands-on exploration of classical cryptography through the full implementation cycle: designing algorithms, handling edge cases such as case preservation and non-alphabetic passthrough, and building cryptanalysis routines to break those same ciphers.

The modular architecture — `utils`, `ciphers_core`, `cryptanalysis`, and two thin driver scripts — kept the codebase readable, testable, and maintainable throughout development. Reusing `ciphers_core` functions inside `cryptanalysis` ensured a single source of truth for all cipher logic.

The central cryptanalysis lesson is that monoalphabetic substitution ciphers (Caesar and Affine) are inherently weak: their key spaces are small enough for exhaustive search in milliseconds. The Transposition cipher is structurally different but equally vulnerable once an attacker can exploit the padding constraint to prune the search space to only the divisors of the ciphertext length. In all three cases, the length-weighted dictionary scoring heuristic correctly identified the original plaintext even in the presence of punctuation and numeric tokens, demonstrating that robust cryptanalysis does not require brittle exact-match validation.